# AI Rescue Mission: An Interactive Grid-Based Demonstration of A* Heuristic Search for Minimum-Cost Rescue Pathfinding

Mukhesh Kumaar R. S. and Yuvaraja A.
Department of Computer Science and Engineering
Rajalakshmi Engineering College, Thandalam, Chennai, India

***Abstract*—**

This paper presents AI Rescue Mission, a browser-based educational system that demonstrates informed search through a grid-based rescue-navigation problem. The system combines an A* pathfinding simulator, a Breadth-First Search (BFS) baseline, an event-level diagnostic trace, and a Player-versus-AI rescue race. Each traversable grid cell is modeled as a state; legal actions move one cell up, down, left, or right, and every legal transition has unit cost. A* ranks frontier states using $f(n)=g(n)+h(n)$, where $g(n)$ is the known cost from the start and $h(n)$ is Manhattan distance to the goal. Under the stated movement model, the heuristic is admissible and consistent. The same map snapshot is supplied to A* and BFS so that route cost and explored-state counts can be compared. On five deterministic simulator challenges, both algorithms returned identical route costs totaling 96 across the five independent maps. A* expanded 147 states while BFS expanded 566, corresponding to a 74.03% aggregate reduction in expanded states; the arithmetic mean of per-map reductions was 73.85%. These measurements indicate lower exploration for the selected maps, not a universal runtime advantage. The implementation executes locally in the browser and does not use a machine-learning model, external AI API, database, or remote pathfinding service. The contribution is an inspectable learning environment that connects search theory to algorithm traces and interactive gameplay rather than a new pathfinding algorithm.

Index Terms—Artificial intelligence, A* search, heuristic search, Manhattan distance, breadth-first search, pathfinding, educational simulation, grid navigation.

## I. INTRODUCTION

Artificial intelligence (AI) includes methods that enable an agent to select actions toward a specified goal. Search is a foundational problem-solving approach: an agent represents possible situations as states, applies legal actions to generate successor states, and continues until it reaches a goal or determines that no solution is available. Navigation tasks provide a concrete setting for examining this process because a route must connect a start position to a destination while respecting environmental constraints.

In a rescue-navigation metaphor, a rover begins at a designated location and must reach a distress beacon while avoiding blocked terrain. A grid makes the environment precise. Each free cell is a state, adjacent free cells are connected by legal transitions, and blocked cells cannot be entered. The route cost is the number of moves required to reach the beacon. This abstraction is useful for teaching search because students can directly relate coordinates, actions, costs, and obstacles to the algorithm's decisions.

A common challenge in teaching heuristic search is the distance between a mathematical definition and the behavior of an executing program. A learner may know that A* uses a heuristic yet still confuse the estimated remaining cost with the path cost already incurred. A learner may also assume that A* simply moves in a straight line toward the goal, overlooking the way obstacles force detours and alter which frontier states are expanded. A useful teaching system should expose not only the final route but also the order in which states are discovered, scored, and expanded.

AI Rescue Mission addresses this challenge through two connected experiences. Its simulator lets a learner inspect A* scores, replay search events, edit a grid, and compare A* with BFS on the same map. Its rescue race turns the computed route into an interactive task in which a human-controlled rover competes against a drone following the A*-computed route. The game is an educational interface over the same search problem; it is not intended to be deployed as a real emergency-response system.

The paper documents the system model, algorithmic choices, browser-local architecture, implementation, deterministic evaluation, and limitations. The evaluation focuses on path legality, agreement in optimal path cost, explored-state counts, and reproducibility of the displayed output. It does not claim a new search algorithm, a statistically validated improvement in learning outcomes, or a universal reduction in execution time.

### A. Problem Statement

Given a finite rectangular grid, a start cell, a goal cell, and a set of impassable obstacle cells, compute a legal minimum-cost route from start to goal. If no route exists, report failure. The system must also expose enough intermediate information for a learner to understand why the algorithm selected a state and how the final path was recovered.

The educational problem is broader than route generation alone. A search simulator must make the difference between frontier states and expanded states visible, present the values of g(n), h(n), and f(n) consistently, and avoid substituting illustrative or hard-coded values for the actual output of the search function. When a baseline is shown, it must receive the same map and use the same definition of an explored state.

### B. Objectives and Contributions

The primary objective is to implement and demonstrate A* search with Manhattan distance on a four-direction, unit-cost grid. A second objective is to provide BFS as an uninformed baseline on identical maps. A third objective is to expose search diagnostics, including event traces, node scores, parent links, and reconstructed routes. A fourth objective is to connect the search output to a simple rescue-race interface without introducing a separate path planner for the drone.

The main contribution is an integrated educational application that combines algorithm execution, visual inspection, controlled comparison, and interactive gameplay. The project applies established algorithms in a shared grid model. Its novelty claim is deliberately limited to this integration and presentation approach; it does not claim to introduce A*, BFS, Manhattan distance, or a new theoretical guarantee.

### C. Scope and Assumptions

The documented simulator supports square grids from 10 × 10 through 15 × 15. Movement is orthogonal, every legal transition has cost one, obstacles are impassable, and the map remains fixed during each search or race. A single beacon is the goal, and the agent has complete knowledge of the supplied grid. These assumptions are important: they make Manhattan distance appropriate and allow BFS to serve as an optimal baseline.

All search and game computations occur in browser memory. The application has no application database, user-account system, machine-learning model, or external AI API. Challenge completion and campaign progress are retained only for the current browser visit. These characteristics make the implementation self-contained but also limit persistence, collaboration, deployment to multiple users, and evaluation of long-term learning progress.

## II. BACKGROUND AND RELATED WORK

A* search is a best-first graph-search algorithm that combines the known cost to a state with an estimate of the remaining cost. The foundational work by Hart, Nilsson, and Raphael formalized heuristic determination of minimum-cost paths and established the basis for A* [1]. The algorithm is commonly introduced through the evaluation function f(n)=g(n)+h(n). The design of h(n), and the conditions under which it is safe to use, are central to both its efficiency and its optimality.

Russell and Norvig describe search as a problem-solving method built from an initial state, actions, a transition model, a goal test, and a path-cost function [2]. These elements map directly onto AI Rescue Mission: the start cell is the initial state, the four directions are actions, grid adjacency defines transitions, the beacon defines the goal test, and the number of moves defines path cost. This explicit mapping helps keep the implementation aligned with the formal problem definition.

Breadth-First Search expands states in increasing depth order when each edge has the same cost. Consequently, BFS finds a shortest path in an unweighted graph, although it may explore many states that are not close to the goal. It provides a useful baseline because the rescue grid uses unit-cost transitions. Comparing A* and BFS under this condition isolates the effect of heuristic-guided exploration more clearly than a comparison in which map structure or edge costs differ.

Red Blob Games provides accessible, interactive explanations of pathfinding and A* implementation details, including graph representation, frontier management, path reconstruction, and visual reasoning [3], [4]. Such material motivates the use of visual search states and trace inspection in an educational interface. The project adapts the broad pedagogical principle of making algorithm behavior visible, while maintaining its own browser implementation and stated scope.

### A. Heuristic Search and Optimality

A heuristic h(n) estimates the minimum remaining cost from state n to a goal. An admissible heuristic never overestimates that true minimum cost. A consistent heuristic satisfies $h(n) \le c(n,n') + h(n')$ for each legal transition from n to n'. In the grid model used here, Manhattan distance is both admissible and consistent because a path must make at least the required number of horizontal and vertical moves, and a single orthogonal move changes Manhattan distance by at most one.

### B. Practical Visualizations

Visualizations can help students connect abstract search operations to visible state changes. However, a visualization must accurately distinguish events from expansions. A trace may include frontier insertions, score updates, duplicate entries, and expansions. Counting all such events as if they were equivalent to expanded nodes can produce misleading comparisons. This paper therefore treats expanded-state count as the principal comparison measure and reports raw event counts separately.

### C. Research Gap

The gap addressed by this project is educational and integrative. The system combines a search simulator, editable grid, node inspector, A*/BFS comparison, and a player-versus-drone race in one browser-local experience. The work does not claim that existing pathfinding algorithms are absent from the literature or that the integration has been shown to improve learning statistically. Instead, it documents an implementation and its measured behavior on five named simulator challenges.

## III. SEARCH PROBLEM FORMULATION

### A. State Space

Let the grid be a finite set of coordinates $S=\{(x,y) \mid 0 \le x < W, 0 \le y < H\}$, where W and H are the grid width and height. Let $O \subseteq S$ be the obstacle set. The traversable state space is $S \setminus O$. The start state s and goal state t must be valid traversable cells. A state is identified by its coordinate pair, and the application stores obstacle coordinates as coordinate keys for membership checks.

For a state n=(x,y), the candidate orthogonal successors are (x+1,y), (x−1,y), (x,y+1), and (x,y−1). A candidate is legal only if it remains within grid bounds and does not belong to O. Diagonal movement is not permitted. This definition gives each free cell at most four immediate neighbors and avoids ambiguity about diagonal corner cutting.

### B. Initial State, Goal Test, and Cost

The initial state is the rover's starting coordinate. The goal test returns true when the current coordinate equals the beacon coordinate. Every legal move has a transition cost of one. For a path P=(s,n1,…,t), its cost is the number of transitions in the path, which equals the number of listed cells minus one. A valid solution must start at s, end at t, remain in bounds, avoid all obstacles, and have Manhattan distance one between every pair of consecutive cells.

$$Cost(P) = |P| - 1$$

If the start equals the goal, the valid route contains one cell and has cost zero. If the frontier becomes empty before the goal is selected, no route exists under the current map constraints. This failure condition is essential because an obstacle arrangement may disconnect the start from the goal.

### C. Search-State Records

Each A* state record contains its coordinate, known path cost g, heuristic h, total score f, parent reference, and ordering metadata. The parent reference points to the predecessor from which the best known score was obtained. When the goal is reached, following parent links backward and reversing the collected sequence reconstructs the route. The reconstructed path is then displayed separately from the broader explored footprint.

The implementation records the open-frontier size and event type as diagnostic information. An open state has been discovered and remains a candidate for later expansion; an explored state has been selected for expansion. These sets are conceptually different. Frontier entries may remain when the goal is reached, and therefore the explored-state count is not necessarily the number of all discovered states.

## IV. A* SEARCH AND HEURISTIC DESIGN

### A. Evaluation Function

A* evaluates each candidate n using the sum of the cost already incurred and the estimated cost remaining. The known cost g(n) is derived from the current best path from the start. The heuristic h(n) estimates the remaining cost to the goal. The combined value f(n) is used to prioritize frontier entries.

$$f(n) = g(n) + h(n)$$

The algorithm initializes the start with g(s)=0 and h(s) equal to its Manhattan distance to the goal. At each iteration, it chooses a frontier entry with the lowest f value. The implementation uses lower h as a secondary tie-breaker and insertion order as a final tie-breaker. These tie-break rules make the selected ordering deterministic for a fixed map and implementation.

### B. Manhattan Distance

$$h(n) = |x(n) - x(t)| + |y(n) - y(t)|$$

Manhattan distance sums the absolute horizontal and vertical coordinate differences. For the report's Easy Rescue example, the start is (1,8) and the goal is (8,1). The estimate is $|1-8|+|8-1|=14$. Since g(s)=0, the initial f score is 14. If a later node has g=3 and h=11, its f score is also 14. Equal f values can therefore arise from states at different positions along a candidate route.

The heuristic ignores obstacle geometry. It is a lower bound because obstacles can require extra moves but cannot remove the horizontal and vertical coordinate changes that are necessary to reach the goal. This distinction is important for learners: h(n) is not necessarily the true remaining route length, and it is not a promise that the drone will follow a visually straight line.

### C. Admissibility

Let h*(n) be the actual minimum remaining cost from n to the goal. Any legal route must change the x coordinate by $|x(n)-x(t)|$ and the y coordinate by $|y(n)-y(t)|$. With only orthogonal moves, at least the sum of these changes is required. Obstacles can increase this minimum but cannot decrease it. Therefore $h(n) \le h^*(n)$, so Manhattan distance is admissible for the specified model.

### D. Consistency

For any legal neighbor n' of n, one orthogonal move changes one coordinate by exactly one and leaves the other unchanged. The Manhattan estimate can therefore change by at most one. With transition cost c(n,n')=1, the consistency inequality $h(n) \le 1+h(n')$ holds. At the goal, h(t)=0. Under the stated model, consistency supports the use of a closed set to avoid expanding a state repeatedly.

### E. Frontier Management

The implementation maintains the open frontier as a sorted array rather than a binary-heap priority queue. Before selecting the next entry, the array is sorted by ascending f, then ascending h, then insertion order. The first entry is removed. A closed set tracks expanded coordinates, while best-known scores and parent references are updated when a lower tentative g value is found.

This design is straightforward to inspect and adequate for the application's small educational grids. Sorting a frontier of k entries costs O(k log k) per sort, and removing the first array entry can require shifting remaining entries. It should not be described as heap-based scaling. A binary heap could reduce priority-selection overhead for larger maps, but it would introduce a different implementation trade-off and should be evaluated separately.

## V. BREADTH-FIRST SEARCH BASELINE

Breadth-First Search is included as an uninformed reference algorithm. It uses a FIFO queue and expands states in the order in which they are discovered. Because every legal move costs one, the first time BFS reaches the goal corresponds to a minimum-move route. The same movement rules and obstacle checks are applied to A* and BFS.

### A. Role in the Evaluation

The baseline provides a comparison for the practical effect of heuristic guidance. If both algorithms receive the same start, goal, grid dimensions, and obstacle set, a difference in explored-state count can be interpreted in relation to their search order, while equal route cost is expected for successful unit-cost cases. The baseline is not intended to show that BFS is incorrect or generally inferior: BFS is a reliable shortest-path method for unweighted graphs.

### B. Fair Comparison Conditions

A fair run uses an identical immutable map snapshot for both algorithms. The start coordinate, goal coordinate, grid bounds, blocked-cell coordinates, legal actions, and edge costs must remain unchanged. Each run records whether a path was found, the returned path cost, and the explored-state count. Timing is measured separately and is not treated as a stable ranking because browser scheduling and device performance can vary.

### C. Event Counts Versus Expanded Nodes

The implementation's A* trace records discovery or update events as well as expansion events. BFS records expansion events. The two trace policies are not identical, so raw trace-event totals are not equivalent units of computational work. On Easy Rescue, A* records 48 events but expands 15 nodes, whereas BFS records 88 events and expands 88 nodes. Expanded-state counts are therefore the primary comparison metric in this paper, while event totals are reported only to explain the visualization.

### D. Expected Behavior

On a map with a relatively direct route, the heuristic can concentrate A* exploration near states with promising f values. In obstacle-heavy maps, a low Manhattan estimate may correspond to a route that is blocked or requires a long detour. A* can consequently expand more states on a difficult map than on an easy map. BFS may explore broad layers around the start without considering goal direction. Neither pattern implies a universal count or runtime relationship for every possible grid.

## VI. SYSTEM ARCHITECTURE

The application has a browser-based simulation layer and a rescue-game layer that share the same grid representation and A* implementation. The grid snapshot contains dimensions, start coordinate, goal coordinate, and blocked coordinates. The simulator consumes search output to render the route and diagnostics; the game consumes the reconstructed route to move the drone. This arrangement reduces the risk that the simulator and game silently use different route logic.

### A. Logical Components

| Component | Responsibility | Output |
|---|---|---|
| Grid / map model | Stores dimensions, start, goal, obstacles | Validated map snapshot |
| Core A* engine | Runs informed graph search | Route, cost, explored count, trace |
| Manhattan helper | Computes goal-distance estimate | h(n) for each evaluated state |
| BFS baseline | Runs FIFO search on same map | Baseline route and explored count |
| Diagnostics / trace | Records events and node scores | Replay, inspector, statistics |
| Rescue game engine | Controls rover, drone, timer, energy | Race state and outcome |
| Field Manual | Explains concepts and formulas | In-application learning reference |

TABLE I. LOGICAL MODULES AND THEIR RESPONSIBILITIES.

The Manhattan calculation is a helper within the search code, not a remote service. BFS is a separate baseline that receives the same grid snapshot. The architecture does not include a database, account service, external AI model, or pathfinding server. Search data remains in memory for the current visit.

### B. Data Flow

The user selects a campaign challenge or edits a custom map. The application validates the grid and represents obstacles as coordinate keys. When A* is run, the engine creates a frontier entry for the start and repeatedly selects the minimum-priority entry. Each legal neighbor is checked, scored, and potentially added or updated. Once the goal is selected, the engine reconstructs the route and returns it with diagnostics. The UI uses this returned object for the path overlay, score inspector, trace replay, and statistics panel.

For comparison, BFS receives the same map snapshot and independently returns its route and explored count. During a race, a fresh A* route is computed at race start. The drone then advances along that precomputed route; it does not launch a new search for each movement. The human rover is controlled independently, subject to boundary and obstacle checks.

## VII. IMPLEMENTATION DETAILS

### A. Technology Stack

The project report identifies React, TypeScript, TanStack Start, and Vite as the interface and application stack. Search data structures include arrays, maps, and sets. The TypeScript search implementation executes as JavaScript in the browser. This stack supports a component-based interface while keeping the pathfinding computation local to the page.

### B. Core A* Procedure

*1) Initialize g(s)=0 and parent(s)=null.*
*2) Insert s into OPEN with f(s)=g(s)+h(s).*
*3) Select the OPEN entry with minimum (f,h,insertion order).*
*4) Skip entries already in CLOSED; otherwise close and record n.*
*5) If n is the goal, reconstruct and return the route.*
*6) For each legal neighbor n', test tentative g(n')=g(n)+1.*
*7) If the score improves, update g, parent, h and f; insert/update frontier.*
*8) Repeat until a route is found or OPEN is empty.*

The outline describes the behavior recorded in the project report. The implementation sorts the open array to choose the minimum entry. A closed set prevents a coordinate from being expanded twice. For each legal orthogonal neighbor, a tentative cost of current.g+1 is calculated. When the cost improves the best-known score, the predecessor and cost are updated. If the frontier is exhausted, the search reports that no route was found.

### C. Route Reconstruction

When the goal is selected, route reconstruction starts at the goal and follows parent references until the start is reached. The collected sequence is reversed to obtain start-to-goal order. This procedure is more robust than trying to infer a route from the visual shape of the explored cells. The application highlights the final path separately from explored states so that a learner can distinguish a solution from the full search footprint.

### D. Input Validation and Legal Moves

Every move candidate must be inside the grid and outside the obstacle set. The game engine rejects rover moves that violate these constraints. A valid returned route is expected to start at the configured start, end at the goal, remain within bounds, avoid blocked cells, and use only orthogonally adjacent cells. These invariants provide a practical basis for smoke testing.

### E. Diagnostics and User Interface

The diagnostics module records the step number, node coordinate, g, h, f, parent, frontier size, and whether the state was added to the frontier or expanded. The simulator can replay the trace at a selected speed or advance one event at a time. The node inspector calculates Manhattan distance for the selected cell and displays g and f when a score has been discovered. The statistics panel reports path availability, route cost, explored states, open states, heuristic name, and measured search duration.

### F. Execution-Time Measurement

The reported duration wraps the search function using a high-resolution timer. It excludes animation, human reaction time, and drone movement delays. A value rounded to 0.00 ms does not mean that the algorithm executes in zero time. Since these are small grids and browser measurements are sensitive to environment, the paper emphasizes deterministic path and node-count measurements rather than claiming a runtime speedup.

## VIII. RESCUE-RACE DESIGN

The rescue-race layer transforms a route-planning problem into a Player-versus-AI activity. The human rover accepts arrow keys, WASD, and on-screen directional controls. The drone follows the A*-computed route. Both agents are constrained by the same map, but they are controlled differently: the rover responds to user input while the drone advances automatically.

### A. Race State Machine

The game state sequence is ready → countdown → racing → over. A fresh A* route is computed when the race begins. Three countdown stages occur at 750 ms intervals. After the race starts, a timer records elapsed time and the drone advances along its precomputed optimal route. Explicit state separation prevents setup controls, countdown timing, movement, and completion handling from interfering with one another.

### B. Player Movement and Energy

Every successful rover move increments the player's step count and consumes one energy unit, including a move back into a previously occupied cell. Invalid moves do not pass through obstacles or grid boundaries. The battery budget is defined as max(8, round(1.6 × optimal cost)+4). For an optimal route cost of 14, this produces 26 energy units. The formula creates a limited movement budget that allows inefficient detours to affect the race outcome.

### C. Campaign and Custom Maps

The documented campaign contains Clear Streets, Collapsed Highway, Urban Ruins Maze, Downtown Gridlock, and Mega Catastrophe. Campaign grids range from 10 × 10 to 15 × 15. Drone movement delays decrease from 620 ms in the first sector to 300 ms in the fifth sector. Custom races allow obstacle editing and random-city generation, with a custom drone delay of 420 ms per move.

### D. Interpreting the Winner

Reaching the beacon ends the race in the player's favor. Drone arrival or exhausted battery produces the corresponding non-player outcome. The post-race view compares the player's movement with the optimal route cost and can reveal the drone's search footprint. A faster human finish does not prove a shorter route because keyboard input rate, reaction time, animation, and configured drone delay affect the winner. The race is designed to make search consequences tangible, not to serve as a controlled benchmark of human navigation.

### E. Accessibility and Feedback

The interface provides keyboard and on-screen rover controls, animation-speed options, and a mute control for browser-generated cues. Move, countdown, start, win, loss, and radar cues can support feedback during gameplay. These controls improve interaction flexibility, although formal accessibility testing and usability evaluation are outside the reported experiments.

## IX. EXPERIMENTAL METHODOLOGY

### A. Evaluation Questions

The evaluation addresses four practical questions: (1) does each algorithm return a legal route when one exists in the supplied challenges; (2) do A* and BFS return equal route costs under the unit-cost model; (3) how many states does each algorithm expand on the five predefined challenges; and (4) do the displayed simulator outputs match independently observed executions of the current implementation? These questions concern algorithm behavior and implementation consistency, not learning outcomes.

### B. Experimental Setup

The results were obtained by executing the current A* and BFS functions on five deterministic simulator challenges. Both functions received identical grid dimensions, start coordinate, goal coordinate, and blocked-cell coordinates. Movement was restricted to four orthogonal directions and every legal edge had cost one. Obstacles were not changed between runs. The reported path cost is the number of edges, equal to returned path length minus one.

### C. Procedure

For each challenge, the obstacle set was constructed, A* was executed, BFS was executed, and the returned paths and explored-state counts were recorded. Successful routes were checked against the map constraints, and their costs were compared. Per-map reduction in explored states was calculated as 100 × (BFS explored – A* explored)/BFS explored. Aggregate reduction was calculated from the sums of explored nodes; the arithmetic mean of the five per-map percentages was calculated separately.

$$\text{Per-map reduction (\%)} = 100 \times (E_{BFS} - E_{A^*}) / E_{BFS}$$

$$\text{Aggregate reduction (\%)} = 100 \times (\Sigma E_{BFS} - \Sigma E_{A^*}) / \Sigma E_{BFS}$$

### D. Verification Conditions

A legal path must begin at the specified start, end at the goal, remain within the grid, avoid blocked cells, and have Manhattan distance one between consecutive cells. The reported output-verification flow opened the application in Chromium at a 1280-pixel-wide desktop viewport, expanded the Field Manual, set animation to Instant, ran A* to completion, and requested same-map BFS comparison. The Easy Rescue display showed path cost 14, 15 A* explored nodes, and 88 BFS explored nodes.

Supplementary source-level smoke checks covered a reachable open grid, a blocked goal, start equal to goal, and basic Manhattan calculations. These checks test ordinary success, frontier exhaustion, the zero-move solution, and basic heuristic values. They do not constitute exhaustive formal verification.

## X. RESULTS

Table II summarizes the five simulator challenges. Both algorithms found a route on every challenge and returned the same route cost. The total cost of 96 is the sum of the five separate route costs; it is not the length of one continuous mission.

| Challenge | Cost | A* expanded | BFS expanded | Reduction |
|---|---|---|---|---|
| Easy Rescue | 14 | 15 | 88 | 82.95% |
| Blocked Route | 15 | 36 | 82 | 56.10% |
| Maze Rescue | 22 | 23 | 113 | 79.65% |
| Multiple Routes | 17 | 44 | 121 | 63.64% |
| Expert Rescue | 28 | 29 | 162 | 82.10% |
| Total | 96 | 147 | 566 | 74.03% |

TABLE II. ROUTE COST AND EXPANDED-STATE COUNTS ON FIVE CHALLENGES.

A* expanded 419 fewer states than BFS across the five maps. The aggregate reduction was (566–147)/566 × 100 = 74.03%. The arithmetic mean of the individual map reductions was 73.85%. The aggregate percentage weights each map according to its BFS explored count, while the arithmetic mean gives each challenge equal weight; they are different summary measures and should not be interchanged.

### A. Per-Map Observations

Easy Rescue had the smallest reported A* explored count: 15, compared with 88 for BFS, with a route cost of 14. Its reduction was 82.95%. This result is consistent with a map where heuristic guidance focuses exploration around states that can lead toward the goal. It should be read as one deterministic case rather than a general performance guarantee.

Blocked Route required 36 A* expansions and 82 BFS expansions, giving a 56.10% reduction. This was the smallest reduction among the five maps. The result illustrates that obstacle placement can reduce the practical guidance offered by a geometric heuristic: Manhattan distance does not encode the detailed shape of a blocked corridor or detour.

Maze Rescue returned cost 22 with 23 A* expansions and 113 BFS expansions, a 79.65% reduction. Multiple Routes returned cost 17 with 44 and 121 expansions, respectively, a 63.64% reduction. These values show that maps with alternative corridors can produce different search footprints even when the route cost remains optimal.

Expert Rescue returned cost 28 with 29 A* expansions and 162 BFS expansions, corresponding to an 82.10% reduction. Its larger route cost did not lead to the largest A* expansion count. Route length alone is therefore insufficient to predict explored-state count; the arrangement of obstacles and the number of competitive frontier states also influence search behavior.

### B. Aggregate Interpretation

The consistent route costs support the expected optimality behavior for the selected unit-cost maps. BFS is optimal for unweighted graphs, and A* uses an admissible, consistent heuristic under the stated model. The observed reductions in expanded states show that A* explored substantially fewer states on these five challenges. However, the experiment includes only five designed maps and does not establish that A* will always expand fewer states, execute faster, or consume fewer resources on every grid.

### C. Trace-Event Results

| Challenge | A* events | BFS events |
|---|---|---|
| Easy Rescue | 48 | 88 |
| Blocked Route | 87 | 82 |
| Maze Rescue | 62 | 113 |
| Multiple Routes | 113 | 121 |
| Expert Rescue | 83 | 162 |

TABLE III. RECORDED TRACE EVENTS; EVENT POLICIES DIFFER.

A* can record more events than BFS even when it expands fewer nodes because its trace includes frontier insertion and update events as well as expansions. Blocked Route is an example: A* recorded 87 events while BFS recorded 82, but A* expanded 36 states compared with BFS's 82. This reinforces the choice to treat expanded-state counts as the principal comparative measure and trace events as a diagnostic display measure.

## XI. DISCUSSION

### A. What the Results Support

The experiments support three limited conclusions. First, the implementation returned a route for all five selected challenges. Second, A* and BFS returned identical costs for these unit-cost maps. Third, A* expanded fewer states than BFS in each of the five recorded cases. The aggregate reduction was 74.03%, and the mean per-map reduction was 73.85%. These are direct observations from the reported dataset.

### B. What the Results Do Not Support

The results do not demonstrate a universal runtime speedup, because expanded-node count and wall-clock time are different measures. They do not show that A* is always more efficient than BFS on every possible map. They do not establish a statistically significant improvement in student understanding, because no controlled learner study or pre/post assessment is reported. Finally, the rescue metaphor does not establish real-world rescue performance: the environment has full map knowledge, static obstacles, deterministic movement, and no sensor uncertainty.

### C. Why Identical Costs Matter

Equal route costs are an important correctness signal, but they are not by themselves a complete proof of correctness. A route could have the right cost while containing an illegal transition, or two algorithms could share a bug. The project therefore also defines explicit path-legality conditions and reports smoke checks. A stronger future evaluation would generate many maps automatically, validate each returned path independently, compare costs with an independent shortest-path oracle, and preserve seeds and raw results for reproduction.

### D. Interpreting Search Efficiency

The heuristic changes the order in which frontier states are selected; it does not remove the need to respect obstacles or prove that the geometrically closest state lies on the final route. The result for Blocked Route, with a smaller reduction than the other maps, illustrates this dependence on map geometry. This is an expected property of heuristic search rather than a contradiction of the heuristic's admissibility.

### E. Educational Value and Its Evidence

The interface makes the search process inspectable through color-coded states, a live node inspector, trace replay, route highlighting, and a post-race radar view. These features plausibly support explanation and exploration, but their educational effectiveness remains a hypothesis until tested with learners. A future study could ask participants to explain g, h, and f before and after using the tool, solve unseen pathfinding problems, and compare results with a control condition. Such a study should report participant selection, instruments, procedure, and statistical analysis.

### F. Engineering Trade-Offs

The sorted-array frontier and array-based BFS queue are simple and transparent for small grids. They may become inefficient as the number of states grows. Replacing the frontier with a binary heap and using an indexed FIFO queue are reasonable engineering improvements, but they should be evaluated with the same test maps and additional larger maps. Implementation changes can alter tie-breaking and trace order even when optimal route costs remain the same.

## XII. LIMITATIONS AND THREATS TO VALIDITY

### A. Limited Map Sample

The evaluation uses five designed maps. These maps demonstrate different challenge labels but are not a random or exhaustive sample of all obstacle distributions. The reported reduction percentages may not generalize to open grids, highly constrained mazes, disconnected regions, or grids with many alternative equal-cost routes.

### B. Metric Limitations

Expanded-state count measures one aspect of search effort. It does not include every data-structure operation, allocation, sort, or rendering cost. The raw trace-event count is especially unsuitable as a cross-algorithm work metric because the event definitions differ. Runtime measurements are reported as implementation diagnostics but are not used to claim a speed advantage.

### C. Implementation-Specific Behavior

The A* implementation uses a sorted array rather than a heap, and BFS uses an array queue. These data structures influence implementation overhead. The tie-breaking policy of lower f, then lower h, then insertion order also affects which optimal path is selected when several equal-cost routes exist. Another correct implementation may return a different route with the same optimal cost.

### D. External Validity

The environment assumes full knowledge of a static grid and a single goal. Real rescue navigation may involve uncertain maps, changing obstacles, physical dynamics, sensor errors, communication delays, safety constraints, multiple objectives, and multiple agents. None of those conditions is modeled by the current project. The term “rescue” is a learning metaphor and must not be interpreted as a field-ready emergency-navigation claim.

### E. Reproducibility

The fixed challenges and map rules support deterministic comparisons, but the paper does not include a public benchmark suite, machine-readable raw trace export, or repeated timing distribution. The UI output was checked in a Chromium desktop viewport, and the report notes no runtime error in the captured flow. This does not guarantee identical behavior across every browser, screen size, or device.

### F. Construct Validity

The paper uses expanded nodes as the primary search-effort measure because it is consistently defined for the reported evaluation. A more complete study would also measure maximum frontier size, number of score updates, memory consumption, and elapsed time under repeated runs. Any such metric should be defined before measurement and applied consistently to both algorithms.

## XIII. FUTURE WORK

### A. Data-Structure Improvements

A binary-heap priority queue could replace repeated sorting of the A* frontier, and an indexed FIFO queue could improve BFS queue handling on larger grids. The resulting implementation should be benchmarked on the existing five maps and a broader generated set. Reporting both node expansions and runtime would help separate algorithmic exploration from data-structure overhead.

### B. Broader Benchmarking

Future experiments should generate maps at multiple sizes and obstacle densities, use fixed random seeds, include reachable and unreachable cases, and run every algorithm on identical snapshots. A route validator should independently check bounds, obstacles, adjacency, start, goal, and cost. The report should publish per-map data and summary statistics, including median and variation in runtime rather than a single timing sample.

### C. Additional Movement Models

Weighted terrain, diagonal movement, or nonuniform transition costs would change the cost model and may require a different heuristic. For diagonal movement, a Manhattan heuristic can overestimate the minimum cost if diagonal steps are allowed at a lower cost than two orthogonal steps. A revised model should define transition costs explicitly and choose a heuristic whose admissibility and consistency can be justified for that model.

### D. Dynamic Obstacles and Replanning

Dynamic obstacles would require the planner to respond when the map changes during execution. The current drone follows a precomputed route and does not replan at each movement. A future version could compare full replanning with incremental search methods, but it would need to define when new observations arrive, how map changes are represented, and how route safety is checked.

### E. Multiple Rescuers

A multi-agent extension could investigate shared flow fields or coordinated planning when several rescuers travel toward one destination. This would be a separate capability, not a feature of the current implementation. The evaluation would need to consider route conflicts, congestion, fairness, and the total cost across agents.

### F. Learning Evaluation and Accessibility

A learner study could assess whether trace inspection improves students' ability to calculate heuristic values, explain frontier order, identify admissibility, and solve unfamiliar search problems. The study should use informed consent and an appropriate academic review process. Additional work could test keyboard-only use, screen-reader descriptions, color-independent state labels, responsive layouts, and the clarity of audio cues.

### G. Persistence and Collaboration

The current application stores campaign progress only for the current visit and does not provide accounts or a database. If persistent progress or collaborative classes become requirements, a backend could be considered. That change would require privacy and security design, data-retention rules, authentication, and a clear explanation of which data leaves the browser. It should not be added merely to make the application appear more complex.

## XIV. CONCLUSION

This paper has described AI Rescue Mission, an interactive browser-local demonstration of heuristic search in a grid-based rescue-navigation setting. The system formalizes the map as a state space, uses Manhattan distance to guide A* search, provides BFS as a unit-cost baseline, and exposes scores, parent links, trace events, and reconstructed routes. A game layer reuses the computed A* route to drive a drone in a Player-versus-AI rescue race.

Across five deterministic simulator challenges, A* and BFS returned identical optimal route costs. The total route cost across the independent maps was 96. A* expanded 147 states compared with 566 for BFS, a 74.03% aggregate reduction and a 73.85% arithmetic mean of per-map reductions. These measurements demonstrate reduced exploration on the selected maps, while leaving universal efficiency and runtime claims unsupported.

The principal contribution is the integration of established search methods into a visible and interactive learning environment. Future work should broaden the benchmark set, improve data structures, independently validate routes over generated maps, measure additional performance metrics, and test educational outcomes with learners. Dynamic terrain, weighted movement, multiple agents, and persistent progress remain possible extensions rather than current capabilities.

## APPENDIX A. ALGORITHM PSEUDOCODE

### A. A* Search

*OPEN ← priority frontier containing start s*
*g(s) ← 0; parent(s) ← null*
*while OPEN is not empty:*
*sort OPEN by (f, h, insertion order)*
*n ← remove first entry from OPEN*
*if n is already CLOSED: continue*
*add n to CLOSED and record expansion*
*if n = goal t: return reconstruct(parent, t)*
*for each legal neighbor n' of n:*
*tentative ← g(n) + 1*
*if tentative improves g(n'):*
*g(n') ← tentative; parent(n') ← n*
*h(n') ← Manhattan(n', t)*
*f(n') ← g(n') + h(n')*
*insert/update n' in OPEN*
*return failure*

### B. BFS Search

*QUEUE ← FIFO queue containing start s*
*mark s discovered; parent(s) ← null*
*while QUEUE is not empty:*
*n ← remove front of QUEUE*
*record expansion of n*
*if n = goal t: return reconstruct(parent, t)*
*for each legal undiscovered neighbor n':*
*mark n' discovered; parent(n') ← n*
*add n' to QUEUE*
*return failure*

The pseudocode is a conceptual summary of the reported behavior. It is not a substitute for the executable source code. Details such as stale frontier entries and exact trace-event fields are implementation-specific.

## APPENDIX B. PATH VALIDATION CHECKLIST

The following checks are recommended for each returned path and can be used in future automated regression tests:

1) Confirm that the path is nonempty when success is reported.

2) Confirm that the first coordinate equals the configured start.

3) Confirm that the final coordinate equals the configured goal.

4) Confirm that every coordinate lies within the grid bounds.

5) Confirm that no coordinate belongs to the obstacle set.

6) Confirm that every consecutive pair differs by one horizontal or vertical unit.

7) Confirm that the reported path cost equals path length minus one.

8) For unit-cost maps, compare A* cost with BFS cost on the same snapshot.

9) For start equal to goal, confirm a zero-cost path containing one cell.

10) For an unreachable goal, confirm that the search terminates with a failure result.

These checks provide practical evidence of implementation consistency. A stronger verification strategy would run them automatically across a large collection of seeded maps and compare returned costs with an independent oracle.

## APPENDIX C. REPORTED EXPERIMENTAL DATA

| Map | Optimal cost | A* expanded | BFS expanded | A* trace events | BFS trace events |
|---|---|---|---|---|---|
| Easy Rescue | 14 | 15 | 88 | 48 | 88 |
| Blocked Route | 15 | 36 | 82 | 87 | 82 |
| Maze Rescue | 22 | 23 | 113 | 62 | 113 |
| Multiple Routes | 17 | 44 | 121 | 113 | 121 |
| Expert Rescue | 28 | 29 | 162 | 83 | 162 |
| Sum | 96 | 147 | 566 | 393 | 566 |

TABLE IV. VALUES TRANSCRIBED FROM THE PROJECT REPORT.

The trace-event totals are sums of the five reported map-level values and are included for reference only. They are not directly comparable measures of algorithmic work because the A* and BFS event policies differ. The sum of route costs represents five separate routes and should not be interpreted as a single combined path.

## ACKNOWLEDGMENT

The manuscript is based on the project report titled “AI Rescue Mission: Find the Best Path Using Heuristic Search,” prepared for the Foundations of Artificial Intelligence course in the Department of Computer Science and Engineering, Rajalakshmi Engineering College. The project report identifies Mukhesh Kumaar R. S. and Yuvaraja A. as the student authors. This research-paper draft preserves the reported experimental values and stated limitations.

## REFERENCES

[1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] A. Patel, “Introduction to the A* Algorithm,” Red Blob Games, 2014. [Online]. Available: https://www.redblobgames.com/pathfinding/a-star/introduction.html. Accessed: Oct. 10, 2026.

[4] A. Patel, “Implementation of A*,” Red Blob Games, 2022 (tutorial page, evolving online resource). [Online]. Available: https://www.redblobgames.com/pathfinding/a-star/implementation.html. Accessed: Oct. 10, 2026.

[5] A. Patel, “Flow Field Pathfinding for Tower Defense,” Red Blob Games, 2014. [Online]. Available: https://www.redblobgames.com/pathfinding/tower-defense/. Accessed: Oct. 10, 2026.

[6] AI Rescue Mission project team, “AI Rescue Mission: Find the Best Path Using Heuristic Search,” project report for the course Foundations of Artificial Intelligence, Department of Computer Science and Engineering, Rajalakshmi Engineering College, Thandalam, Chennai, India, 2026.

## APPENDIX D. COMPLEXITY AND RESOURCE CONSIDERATIONS

### A. Graph Size

For a grid of width W and height H, the maximum number of cells is WH. Obstacles reduce the number of traversable states. Under four-direction movement, the number of legal undirected adjacency relationships is bounded by the horizontal and vertical neighboring pairs, giving at most $W(H-1)+H(W-1)$ edges in a fully open rectangular grid. The actual number is lower when obstacles remove cells or block adjacencies.

### B. BFS Complexity

With a conventional adjacency-list representation and an efficient FIFO queue, BFS runs in $O(V+E)$ time and $O(V)$ auxiliary space, where V is the number of traversable states and E is the number of legal transitions. The project's array-based queue remains conceptually FIFO; however, exact implementation overhead depends on whether the queue removes its first element by shifting array entries or uses an index into the array. The report identifies an array queue but does not provide enough detail to assert a more specific measured complexity for its exact implementation.

### C. A* Complexity

A* complexity depends on the heuristic, graph structure, and frontier implementation. In this project, the open array is sorted at each selection step. Sorting k entries is $O(k \log k)$, and removing the first entry may shift the remaining elements. Consequently, the implementation's overhead is not the same as a heap-based A* implementation. It is appropriate for the small grids used for instruction, but performance on much larger state spaces should be measured rather than inferred from the five-map node counts.

### D. Space Use

Both algorithms maintain records for discovered states, parent relationships, and search bookkeeping. The A* frontier can contain candidate entries that have been discovered but not yet expanded, while the closed set records expanded coordinates. The exact peak memory footprint depends on map topology and data-structure details. The current evaluation does not report memory usage, so no numerical memory advantage is claimed.

### E. Why Fewer Expansions Is Not Identical to Faster Execution

A* may expand fewer states while performing more work per expansion due to heuristic evaluation, frontier ordering, and score updates. Conversely, BFS may have simpler per-node operations but visit a larger portion of the grid. Runtime is determined by both the amount of exploration and the cost of each operation, together with the runtime environment. A node-count reduction is therefore an interpretable search-effort result, not a substitute for repeated wall-clock benchmarks.

## APPENDIX E. REPRODUCIBILITY AND REPORTING TEMPLATE

A reproducible experiment should identify the exact map dimensions, start and goal coordinates, obstacle coordinates or a stable map identifier, algorithm version, tie-breaking rule, and definition of an expansion. For every run, it should record success or failure, route coordinates, route cost, expanded-state count, peak frontier size, number of score updates, trace-event count, and elapsed search duration. If timing is reported, the environment and number of repeated trials should also be stated.

| Field | Recommended value to record |
|---|---|
| Map identity | Name plus obstacle-set hash or explicit coordinates |
| Grid dimensions | Width × height |
| Movement model | Four-direction, unit-cost |
| Algorithm | A* or BFS; version/commit |
| A* heuristic | Manhattan distance; tie-break policy |
| Outcome | Path found / no path |
| Path validation | Bounds, obstacles, adjacency, endpoints |
| Search effort | Expanded states, frontier peak, updates |
| Timing | Repeated durations and summary statistics |
| Environment | Browser, OS, viewport, device details |

TABLE V. SUGGESTED REPRODUCIBILITY RECORD.

The current report provides the five challenge names, route costs, explored counts, and trace-event counts, together with a description of the desktop output-verification flow. It does not provide complete obstacle coordinate lists or a machine-readable export of all traces in the extracted text. Those

omissions limit independent reconstruction of every exact challenge from the paper alone.

## APPENDIX F. ETHICAL AND PRACTICAL BOUNDARIES

The application is an educational demonstration and should be presented as such. Its use of rescue terminology creates an engaging scenario but does not imply that it is suitable for operational emergency planning. Real deployment would require extensive validation, robust sensing, uncertain-state reasoning, safety constraints, fault handling, and assessment under realistic conditions. The current model assumes that the full map is known and static.

The project does not require users to provide personal data to run the local search logic, and it has no account system or database according to the report. If future versions introduce analytics, accounts, saved maps, or online collaboration, the developers should document what is collected, why it is collected, how long it is retained, and how users can remove it. Privacy and security requirements should be treated as design constraints rather than added after implementation.

For classroom use, the trace and score display should be explained carefully. The system should distinguish heuristic estimates from actual costs, and explored cells from final-route cells. It should also disclose when a value is unavailable because a state has not been discovered. Clear terminology helps prevent a visual interface from reinforcing incorrect mental models.

## APPENDIX G. FORMAL STATE-SPACE MODEL

The pathfinding task can be represented as a tuple (S, A, T, s, G, c). S is the set of traversable grid cells; A is the set of movement actions; T maps a state and action to a successor when that action is legal; s is the initial cell; G is the goal condition; and c is the transition-cost function. For the current implementation, a state is a coordinate pair, the action set contains four orthogonal directions, the goal condition tests coordinate equality, and each legal transition costs one.

Separating the model from the algorithm is useful because A* and BFS should solve the same underlying problem. If the obstacle representation or movement rules differ between the algorithms, a comparison of explored counts is not interpretable. The map snapshot therefore needs to be treated as immutable during an individual run. The simulator and game can render the same model differently, but they should not silently redefine which transitions are legal.

A state-space model also clarifies edge cases. If the start equals the goal, the solution is the one-state sequence containing that cell, with cost zero. If the goal is blocked or disconnected, a complete search should terminate without a route. If a neighbor lies outside the grid, it is not a member of the legal successor set. These conditions should be tested explicitly rather than left to the visual interface.

The model is intentionally deterministic and fully observable. A coordinate always represents the same location during a run, and the agent knows the complete obstacle set. This is a useful teaching abstraction, but it is not a realistic model of every rescue environment. Unknown terrain, moving obstacles, uncertain localization, and changing goals would require additional state or planning mechanisms.

## APPENDIX H. HEURISTIC DESIGN DECISIONS

A heuristic should be selected with reference to the transition model rather than by visual intuition alone. In a four-direction grid with unit movement cost, Manhattan distance is the sum of the horizontal and vertical coordinate differences. It gives a lower bound on the number of required moves because each action changes exactly one coordinate by one unit. Obstacles can add detours, but they cannot make the coordinate difference disappear.

Euclidean distance is the straight-line geometric distance between two points. It can be useful for models that allow movement in arbitrary directions, but its numerical value does not directly count orthogonal unit steps. Chebyshev distance is the maximum of the absolute coordinate differences and is often relevant when diagonal movement is allowed at a comparable cost. The Field Manual compares these distance concepts for education, while the operational A* engine continues to use Manhattan distance.

Changing the movement model changes which heuristic is appropriate. If diagonal moves are introduced, the cost of a diagonal step must be specified. If diagonal movement costs one, Manhattan distance may overestimate the true remaining cost, because a diagonal step can reduce both coordinate differences at once. If terrain has variable costs, a distance estimate may need to be multiplied by a valid lower bound on the cost per move. These choices require a new correctness argument.

An admissible heuristic never overestimates the true optimal remaining cost. Consistency is a local condition over each edge: the estimate at the current node cannot exceed the edge cost plus the estimate at its successor. For the current model, Manhattan distance satisfies both properties. Those properties explain why the algorithm can prioritize promising nodes without sacrificing optimality under the assumptions described in the paper.

## APPENDIX I. TIE-BREAKING AND DETERMINISM

Multiple routes can have the same minimum cost in a grid. A search algorithm may therefore return different valid paths depending on the order in which neighbors are generated, how equal-priority frontier entries are handled, and whether duplicate entries are retained or replaced. A different route shape is not necessarily a correctness defect when the route costs are equal and all path constraints are satisfied.

The reported A* implementation orders frontier entries by ascending f score, then ascending h score, then insertion order. The secondary h criterion tends to prefer entries estimated to be closer to the goal when f values tie. Insertion order provides a deterministic final criterion for entries that share both values. This policy is an implementation choice rather than a requirement of A* itself.

Deterministic tie-breaking improves repeatability for the same implementation and map snapshot. It makes screenshots, trace replays, and output verification easier to reproduce. It does not guarantee that a different browser build or a rewritten implementation will return the same route sequence, because neighbor ordering or data-structure behavior may differ.

To reproduce a run, an experiment should record the map, the algorithm version, and the tie-breaking policy. When route equality is not required, tests should compare route validity and optimal cost rather than demand byte-for-byte identity of the route. If a project requires a canonical route, that requirement should be specified explicitly and tested separately from optimality.

## APPENDIX J. OPEN, CLOSED, AND PARENT RECORDS

The open frontier contains candidates that have been discovered but have not yet been expanded. The closed set records coordinates selected for expansion. A coordinate can be discovered before it is expanded, and the search can stop while some candidates remain in the frontier. For this reason, the number of open entries, number of discovered states, number of expanded states, and number of trace events are different quantities.

The best-known g score stores the lowest discovered path cost from the start to a coordinate. If another predecessor provides a lower tentative cost, the record must be updated. The parent pointer is changed at the same time so that route reconstruction follows the predecessor corresponding to the improved score. Failing to update parent data when the score improves can cause the returned route to disagree with the cost used during search.

Parent pointers create a compact representation of a candidate route tree. They avoid copying an entire route for every frontier entry. Once the goal is reached, following parent pointers from the goal back to the start yields the route in reverse order. Reversing this sequence provides the path for display and for the drone's movement.

Diagnostic views should distinguish a coordinate that has never been discovered from a coordinate whose scores are known. An empty g or f value for an undiscovered cell is not a zero score; it means that the algorithm has not yet assigned a recorded path estimate to that state. This distinction helps learners avoid interpreting missing values as numerical results.

## APPENDIX K. TEST DESIGN FOR SEARCH CORRECTNESS

A practical test suite should cover ordinary success and edge cases. A simple open grid checks whether the search can find a direct route. A map with a detour checks obstacle handling. A maze checks repeated branching and dead ends. A blocked goal checks that an invalid destination is rejected or reported as unreachable according to the interface contract. A disconnected map checks frontier exhaustion.

The start-equals-goal case verifies that the system can return a zero-cost solution without attempting a movement. Boundary cases should place the start or goal near each grid edge and verify that successor generation never wraps from one row to another or moves outside the map. These tests are especially important when coordinates are encoded as strings or flattened indices.

Every successful path should be validated independently of the search function. The validator should confirm that the route begins and ends at the configured coordinates, each coordinate is within bounds, no coordinate is blocked, and each adjacent pair differs by exactly one in Manhattan distance. The reported path cost should equal the number of transitions. For unit-cost maps, A* and BFS costs should agree.

Property-based testing could generate many small grids with fixed random seeds and apply the same checks. For every reachable case, compare the A* route cost with an independent BFS result. For every unreachable case, confirm that both algorithms report failure. This is stronger than checking a handful of screenshots, although it still does not constitute a formal proof of the implementation.

## APPENDIX L. EXPERIMENTAL METRIC DEFINITIONS

An evaluation is only meaningful when its metrics are defined consistently. In this project, an expanded state is a coordinate selected for expansion after duplicate or closed-entry handling. The explored count includes the goal expansion, as specified in the project report. The definition should remain fixed when comparing algorithms or when reporting results from future versions.

Trace events are a different metric. The A* trace records discovery or update events and expansion events, while BFS records expansion events. Their totals therefore reflect different logging policies. It would be possible to harmonize the trace schema by recording the same event categories for both algorithms, but such a change would need to preserve the distinction between search operations and display events.

Route cost is the number of edges in the returned path. Since every legal transition costs one, the route cost equals the number of path cells minus one. The aggregate route cost in the results table is a sum across five separate maps and is not a single path length. This detail matters when interpreting summary tables and abstracts.

Node-count reduction is calculated relative to BFS for each map. The aggregate reduction uses summed node counts before division; the arithmetic mean averages the five per-map percentages. The two results answer different questions. Aggregate reduction is influenced more by maps where BFS expands many states, while the arithmetic mean weights each challenge equally.

## APPENDIX M. INTERPRETING THE FIVE MAPS

Easy Rescue has a reported optimal cost of 14 and expands 15 states with A*, compared with 88 for BFS. This large difference illustrates a case where the heuristic helps prioritize states associated with the goal direction. It does not show that the same ratio will hold on all open or nearly open grids, because tie-breaking, obstacle layout, and goal position can change the search footprint.

Blocked Route has cost 15, with 36 A* expansions and 82 BFS expansions. Its 56.10% reduction is the smallest of the five reported reductions. A blocked corridor or forced detour can make Manhattan distance less informative about the true route. The heuristic remains a lower bound, but several states that look promising by coordinate distance may lead to constrained paths.

Maze Rescue has cost 22 and reports 23 A* expansions against 113 BFS expansions. Multiple Routes has cost 17 and reports 44 against 121. The contrast suggests that route cost alone does not determine the number of states expanded. Branching structure, obstacle placement, and the number of frontier states with competitive f scores influence exploration.

Expert Rescue has the highest reported route cost, 28, but A* expands 29 states, fewer than on Blocked Route and Multiple Routes. This is another reason not to infer search effort from route length alone. These interpretations are qualitative and map-specific; complete obstacle coordinates would be needed to reconstruct the exact geometry from the paper alone.

## APPENDIX N. READING A SEARCH TRACE

A trace is a time-ordered record of selected search events. It can show which coordinate was processed, what its current scores were, which parent led to it, how many entries were in the frontier, and whether the event represented a discovery, update, or expansion. A trace is useful because it lets a learner relate a visible change on the grid to a specific operation in the algorithm.

At the start of A*, the start coordinate has g equal to zero and h equal to its Manhattan distance from the goal. Its f value equals h. When a neighbor is discovered, its tentative g value is one greater than the current node's g under the unit-cost model. Its heuristic is calculated from the neighbor coordinate, and f is the sum. Other frontier nodes may have the same f score, so tie-breaking determines which is selected next.

An event trace should not be mistaken for a full list of distinct states. A node may be inserted, updated, and later expanded. Duplicate entries may be skipped if their coordinate is already closed. The trace can therefore contain more rows than there are expanded states. Conversely, a trace policy that only records expansions will have a closer relationship to the explored count.

Trace replay speed affects how quickly a learner sees the operations but does not alter the underlying deterministic route result. If the same map and algorithm are used, switching the animation to Instant should affect presentation duration, not path cost or explored-state count. A reproducibility check can use this expectation to detect accidental coupling between animation and search execution.

## APPENDIX O. INTERFACE DESIGN AND LEARNER FEEDBACK

The simulator presents multiple kinds of information: the grid, the final route, explored states, frontier states, node scores, trace rows, and aggregate statistics. These displays should use consistent terminology and visual encoding. The final route should not be confused with every explored cell, and a frontier cell should not be presented as if it were already expanded.

The node inspector provides a local explanation for a selected cell. Manhattan distance is calculated from the selected coordinate to the goal. If the algorithm has discovered the cell, the interface can show g and f; if it has not, those values should remain unavailable. This supports the distinction between a geometric property of a cell and a path-dependent property discovered by the algorithm.

The Field Manual provides a reference for the problem formulation, equations, pseudocode, applications, and learning outcomes. Keeping explanation close to the interactive display can help learners test a concept immediately after reading it. However, the presence of a manual and visual feedback is not evidence by itself that learning has improved. That claim requires a study with defined outcomes and an appropriate comparison.

Feedback should avoid implying that the drone is using machine learning or an external AI service. According to the project design, the drone follows a route computed by A* in the browser. Accurate language matters because the project teaches search algorithms, not model training or autonomous robotic perception.

## APPENDIX P. PERFORMANCE EVALUATION PLAN

A future performance study should separate algorithmic work from implementation overhead. Expanded-state count captures search exploration, while elapsed time includes data-structure operations and runtime behavior. Peak frontier size gives an indication of memory pressure, and score-update counts describe how often the best-known path to a state improves. No single metric fully characterizes search performance.

Experiments should include repeated trials under a documented environment. Deterministic path and node-count results may remain fixed for a map, but wall-clock measurements can vary due to scheduling, browser load, CPU power management, and background processes. A robust report should publish the median and spread of repeated measurements rather than treating a single very small duration as decisive.

Benchmark sets should vary grid size, obstacle density, start-goal separation, corridor structure, and the presence of multiple equal-cost paths. Unreachable maps should be included to test the cost of exhaustive failure. Each generated map should have a stable seed or a saved obstacle list so that a result can be reproduced.

The comparison should keep all non-algorithmic conditions equal. Both algorithms should receive the same immutable map, and their outputs should be checked by an independent validator. If a heap-based A* implementation is introduced, it should be evaluated as a separate implementation variant. This would help distinguish benefits from the heuristic from benefits due to a data structure.

## APPENDIX Q. POSSIBLE HEURISTIC EXTENSIONS

Manhattan distance is a natural fit for the current orthogonal grid. A future system could offer multiple movement models as separate controlled experiments, but it should never switch heuristics without explaining the change in assumptions. The selected heuristic must be compatible with the transition costs and legal actions.

For a grid allowing diagonal movement with unit diagonal cost, Chebyshev distance is a lower-bound candidate because a diagonal step can reduce both coordinate differences. If diagonal moves cost the square root of two while orthogonal moves cost one, an octile-style estimate can better reflect the permitted motion. The precise formula depends on the exact edge-cost model, so a paper should state the assumptions rather than naming a heuristic without definition.

Weighted terrain requires the cost of each transition to be included in g. If all step costs are at least a known positive minimum, a geometric lower bound may be multiplied by that minimum to remain admissible. If the terrain contains zero-cost transitions or special teleports, simple coordinate distance may no longer be a valid lower bound.

Alternative heuristics can be compared on solution cost, expansion count, and computational overhead. The comparison should include a proof or argument for admissibility and consistency where optimality is claimed. The current paper's experimental values apply only to the implemented Manhattan heuristic and should not be reused as results for hypothetical extensions.

## APPENDIX R. DYNAMIC MAPS AND ONLINE PLANNING

The present system assumes that obstacles remain fixed throughout a search and race. This assumption allows the route to be computed once and then followed by the drone. If an obstacle appears after the route is computed, the drone may attempt to follow a path that is no longer legal unless the game checks and responds to map changes.

Online planning introduces additional design decisions. The application must specify how new information is sensed, when the map is updated, whether the current state is trusted, and what happens if the goal becomes unreachable. It must also define whether the planner restarts from scratch or repairs a previous search result. These decisions affect both correctness and the user experience.

A simple future version could stop the drone when a route cell becomes blocked, update the map, and run A* again from the drone's current coordinate. This is easy to explain but may repeat substantial work. Incremental search methods may reuse information across changes, but their implementation and correctness require separate treatment.

Any dynamic-map experiment should report how often changes occur, whether they are random or scripted, and whether both compared methods receive the same information at the same time. Without this control, differences in route quality or runtime may reflect different observations rather than the planner itself.

## APPENDIX S. MULTI-AGENT AND SHARED-GOAL EXTENSIONS

Multiple rescuers travelling toward one destination create a different planning problem from a single-agent path. A shared flow field can assign each traversable cell a direction or next step that guides several agents toward a common goal. Such a field may avoid running a complete independent search for every agent, depending on the model and update requirements.

Coordinated movement introduces interactions absent from the current system. Two agents may attempt to enter the same cell, block each other in a narrow corridor, or create congestion near the goal. A multi-agent system must decide whether collisions are forbidden, whether agents can wait, and how priorities are assigned. It must also define whether the objective is to minimize total travel, the time until the last agent arrives, or some other measure.

The current project does not implement a shared flow field or coordinated multi-agent planner. The existing reference to flow-field pathfinding motivates a possible extension, not a claim about a feature already present. A future study should implement the alternative and compare it with independent A* planning on reproducible scenarios.

Evaluation metrics could include total path cost, maximum arrival time, conflicts, number of replans, and computation time. These metrics should be defined before experiments begin. A comparison should also state whether agents have identical goals and whether the environment is static or dynamic.

## APPENDIX T. SECURITY, PRIVACY, AND DEPLOYMENT NOTES

The current project is browser-local and does not include a database, account system, external AI API, or remote pathfinding service. This reduces infrastructure requirements for the search demonstration and means that the core algorithm can run without sending the map to a backend. It does not automatically establish that every deployment choice is risk-free; the page may still be served by a hosting provider, and browser behavior depends on the surrounding application.

If persistent progress or user accounts are added, the design will need authentication, access control, data validation, retention rules, and a clear privacy notice. A saved custom map may reveal information about a user or institution depending on its contents. The application should collect only what is necessary for the intended feature and should make persistence explicit.

Deployment testing should check that the page loads over the intended hosting configuration, keyboard controls work, the grid remains responsive at different viewport sizes, and the search still produces deterministic outputs. Browser-console errors, accessibility checks, and automated route-validation tests should be included in a release checklist.

The educational rescue metaphor should also be communicated responsibly. The application is not a certified emergency navigation tool. It assumes a known static map and does not account for real sensor data, uncertainty, physical dynamics, safety protocols, or operational constraints. This boundary should remain clear in documentation and demonstrations.

## APPENDIX U. PROPOSED LEARNER STUDY

A learner study could test whether interactive traces help students understand heuristic search more effectively than reading equations and pseudocode alone. The study should define a target population, recruitment process, consent procedure, learning objectives, and assessment rubric before data collection. The project report does not contain such a study, so no learner-performance result is claimed in this paper.

A possible design is to administer a short pre-test covering state-space formulation, g/h/f interpretation, Manhattan distance, and optimality. Participants could then use the simulator under a defined set of tasks, followed by a post-test containing both familiar and transfer problems. A comparison group could receive a conventional explanation with the same instructional time. The assignment should be balanced to avoid giving one group more practice.

Assessment should measure conceptual explanation as well as numerical answers. For example, a participant might be asked why an obstacle can force a detour even when Manhattan distance decreases, or why two algorithms can return different route shapes but the same optimal cost. A rubric should define partial credit and be applied consistently, ideally with independent scoring for a sample of responses.

Ethical reporting would include the number of participants, exclusions, missing data, study limitations, and analysis method. The study should not imply causal effectiveness from informal comments alone. Results should be reported only after data are actually collected and analyzed.

## APPENDIX V. IMPLEMENTATION CHANGE CONTROL

Search behavior can change when a developer modifies neighbor ordering, tie-breaking, duplicate handling, route reconstruction, or the definition of an expansion. Such changes may leave the final route cost unchanged while altering the trace and explored count. This is why implementation versioning and regression tests are important when reporting numerical results.

A release should preserve the challenge maps and the reported baseline outputs as regression fixtures. A test can verify that the Easy Rescue challenge returns cost 14, A* expands 15 states, and BFS expands 88 states for the documented version. If a future change alters these values, the developer should determine whether the change is expected and update the paper rather than silently retaining old measurements.

Not every implementation change should be rejected because it changes a count. A new priority queue may choose a different equal-cost route or change the order of expansions. The test should distinguish invariant properties—path legality and optimal cost—from version-specific observations such as exact trace order and explored-node count.

A clear changelog can identify the code version used for an experiment. If source code is shared, a stable commit identifier and map fixtures make the results easier for another researcher to reproduce. The current report does not include a commit identifier in the extracted content, so this remains a recommended improvement.

## APPENDIX W. SUMMARY OF CLAIMS AND EVIDENCE

The strongest claims in this manuscript are directly tied to the documented implementation and five-map evaluation: the model uses orthogonal unit-cost moves; A* uses Manhattan distance; BFS is run on the same map snapshot; both methods returned equal costs for the five reported challenges; and A* expanded fewer states in those cases. The aggregate node-count reduction is calculated from the supplied totals.

Claims about educational effectiveness are intentionally framed as motivation or future work. The interface provides features that could support learning, including trace replay and score inspection, but the report does not provide a controlled learner study. Claims about real-world rescue performance are excluded because the environment omits the sensing, uncertainty, dynamics, and safety constraints required for such a conclusion.

Claims about universal algorithm superiority are also excluded. The observed reductions apply to the five selected maps. A* can still explore many states on difficult obstacle layouts, and lower expansion count does not guarantee lower wall-clock time for every implementation. The sorted-array frontier is an acknowledged trade-off.

Keeping a visible boundary between observations, interpretation, and future proposals strengthens the credibility of the work. It lets readers understand what was implemented, what was measured, what follows from standard search theory, and what remains untested.

## APPENDIX X. RESEARCH REPRODUCTION CHECKLIST

Before repeating the experiment, confirm the exact challenge version and whether the grid, start, goal, or obstacles have been edited. Use the same snapshot for A* and BFS. Record the browser and viewport, but do not treat a single desktop run as proof of cross-device compatibility. If screenshots are captured, label them as application outputs and record the challenge name and run settings.

During each run, record whether a path was found, the returned sequence of coordinates, path cost, expanded-state count, frontier size if available, and trace-event count. Validate the path independently. If runtime is included, repeat the measurement and report the spread. Keep animation and game movement delays separate from the measured search function duration.

After running all maps, compute each per-map reduction using the same formula and calculate the aggregate reduction from summed counts. Compute the arithmetic mean of per-map percentages separately. Check the arithmetic before publication. Explain that summed route costs represent separate paths and that raw event counts are affected by logging policy.

Finally, archive the map fixtures and software version used. Record deviations from the documented procedure. If the output differs from the report, investigate whether the map, algorithm, tie-breaking, or event semantics changed. This checklist is intended to make future evaluations more transparent and repeatable.